Python

Basic String

using placeholder

```
x = 10
s = "Hello world %d"%x
```

using formatting

```
x = 12
s = f"Hello world {x}"
```

Logging

```
import logging
import numpy as np

logging.basicConfig(filename='logfile.log',
                                format='%(asctime)s %(levelname)s: %(message)s',
                                filemode='w')

logger = logging.setLogger()
logger.setLevel(logging.DEBUG)
x=np.random.random((2,3,4))
logger.debug(f'x :{x}')
logger.info("Just an information")
logger.warning("Its a warning")
logger.error("Did you try to divide by zero?")
logger.critical("Internet is down")
```

Numpy

np.random.rand(2,3,4) -> standard uniform
np.random.random((2,3)) -> random_sample from standard uniform
np.random.uniform(0,5,(2,3)) -> non-standard uniform distribution

np.random.randn(2,3,4) -> standard normal
np.random.normal(0,5,(2,3)) -> non-standard normal distribution

```
$a1 = np.random.rand(5,10,2)
$b1 = np.random.rand(5,5,2)
$np.hstack((a1,b1)).shape
(5,15,2)
$np.concatenate((a1,b1),axis=1)
a2 = np.random.rand(10,5,2)
b2 = np.random.rand(5,5,2)
$np.vstack((a2,b2)).shape
(15,5,2)
$np.concatenate((a2,b2),axis=0).shape
(15,5,2)
```

np.nonzero(J) -> returns tuples of indices of J that are nonzero
np.maximum(x1,x2) -> returns he maximum of *x1* and *x2*, element-wise. This is a scalar if both *x1* and *x2* are scalars.
np.max(J, axis=1) -> Maximum of *a*. If *axis* is None, the result is a scalar value. If *axis* is an int, the result is an array of

dimension `a.ndim - 1`. If *axis* is a tuple, the result is an array of dimension `a.ndim - len(axis)`.
np.where(J>t,J,0) -> fills the position where the boolean condition is true with elements of J, 0 otherwise.

Get indices of N maximum values in a numpy array:

```
n=5
arr=np.random.rand(2,3)
x, y = np.unravel_index(np.argsort(arr.ravel(), axis=None)[::-1][:n], arr.shape)
```

Matplotlib

```
# Plot the images in the batch, along with the corresponding labels
fig = plt.figure(figsize=(10, 4))
# Display 20 images
for idx in np.arange(27):
    ax = fig.add_subplot(3, 9, idx + 1, xticks=[], yticks=[])
    imshow(example_data[idx].detach().numpy())
    ax.set_title(classes[example_targets[idx]])
plt.tight_layout()
plt.show()
```

```
# Plot the images in the batch, along withe the corresponding labels
fig, subs = plt.subplots(2, 10, figsize=(25, 4))
for idx, sub in zip(np.arange(20), subs.flatten()):
    sub.imshow(np.squeeze(images[idx]), cmap="gray")
    sub.set_title(str(labels[idx].item()))
    sub.axis("off")
plt.savefig("MNIST-data.png")
plt.show()
```